Treinamento e Validação YOLO

1. Introdução

Para o desenvolvimento de um sistema capaz de identificar automaticamente placas de veículos em imagens, foi utilizado um modelo baseado na arquitetura **YOLOv8**, reconhecida por sua eficiência em tarefas de detecção em tempo real. O treinamento e a validação foram realizados com um conjunto de dados customizado conforme definido no arquivo `data.yaml`, contendo as imagens anotadas de veículos e suas respectivas placas.

2. Arquitetura Utilizada

Foi adotada a versão **YOLOv8n (nano)**, escolhida por três motivos principais:

1. **Baixo custo computacional** – ideal para treinamento em CPU e implantação em sistemas embarcados ou com baixa capacidade de hardware.
2. **Velocidade** – mesmo sendo leve, entrega inferências rápidas, úteis em sistemas de monitoramento.
3. **Desempenho competitivo** – mantém boa acurácia em problemas de detecção de objetos pequenos, como placas.

O modelo pré-treinado (`yolov8n.pt`) foi usado como base para fine-tuning, aproveitando o *transfer learning* para acelerar a convergência.

3. Procedimento de Treinamento

O treinamento foi realizado com o seguinte script:

```
from ultralytics import YOLO
import torch

model=YOLO("yolov8n.pt")
```

```
results=model.train(
data="data.yaml",
epochs=120,
imgsz=640,
batch=4,
device="cpu",
name="placa_detector"
)

print("Treinamento finalizado")
```

Justificativas das escolhas:

- **epochs = 120**
  Permitiu que o modelo convergisse adequadamente, especialmente considerando o uso de CPU, que tende a treinar mais lentamente. Quantidades menores não atingiam estabilização da perda (*loss*).
- **imgsz = 640**
  Resolução padrão recomendada pela própria biblioteca.
  → Placas são objetos pequenos; resoluções menores perdem detalhes importantes.
- **batch = 4**
  Ajustado ao limite de memória da máquina utilizada, garantindo estabilidade no treinamento em CPU.
- **device = "cpu"**
  Treinamento realizado sem GPU, exigindo escolhas mais leves de modelo e configuração.
- **name = "placa_detector"**
  Organização dos experimentos dentro da pasta `runs/detect/`.

Por que YOLO para placas?

- Detecta objetos pequenos **melhor que métodos tradicionais**.

- Entrega **resultados em tempo real**, ideal para monitoramento de tráfego.
- Facilita pipeline posterior com OCR (como EasyOCR ou Tesseract).
- Suporta perfeitamente *transfer learning*, essencial quando o dataset é limitado.

4. Procedimento de Validação

A validação foi feita com o script:

```
from ultralytics importYOLO

model=YOLO("runs/detect/placa_detector/weights/best.pt")

metrics=model.val(
data="data.yaml",
split="test",
imgsz=640
)

print("\n===== RESULTADOS =====")
print("mAP50:",metrics.box.map50)
print("mAP50-95:",metrics.box.map)
print("Precisão:",metrics.box.mp)
print("Recall:",metrics.box.mr)
```

Objetivo da validação

- **Verificar a capacidade real do modelo** de detectar placas em imagens nunca vistas.
- Avaliar métricas essenciais:
  - **mAP50** – mede a precisão média considerando IoU de 0.50, útil para objetos pequenos como placas.
  - **mAP50–95** – métrica mais rigorosa (padrão COCO).

- **Precisão (Precision)** – indica o quanto das detecções feitas realmente eram placas.
- **Recall** – indica o quanto o modelo deixou de detectar.

Por que validação independente?

Para evitar *overfitting*, garantindo que o modelo generaliza para diferentes condições:

- variados tipos de veículos;
- diferentes iluminação e ângulos;
- placas sujas, inclinadas ou parcialmente obstruídas.

5. Justificativa Geral da Abordagem

Recorrer ao YOLO e ao treinamento supervisionado foi necessário porque:

1. **Placas possuem variação realista** (tamanho, cor, ângulo, distância, iluminação).
   Métodos clássicos (Canny, Haar Cascade, Contornos) falham em cenários complexos.
2. **O modelo aprende características visuais robustas**, como:
   - bordas retangulares específicas,
   - padrões de alto contraste,
   - posição típica em veículos,
   - proporção largura/altura da placa.
3. **O sistema precisa ser eficiente** o bastante para integrar:
   - câmeras de monitoramento,
   - raspagem automática,
   - OCR,
   - automação de estacionamentos e portarias.

4. **Treinar um modelo específico para placas brasileiras** melhora muito a detecção em comparação a modelos genéricos.

6. Conclusão

O processo de treinamento e validação descrito permitiu a criação de um modelo YOLO especializado na detecção de placas veiculares.

As configurações adotadas foram definidas considerando limitações de hardware, características do dataset e a necessidade de precisão adequada em objetos pequenos.

Os resultados obtidos na validação garantem que o modelo está apto para uso em pipelines de reconhecimento de placas (LPR/ANPR), podendo futuramente ser integrado a um sistema OCR para extração do texto.